

12

Applicative and traversable functors

In the previous chapter on monads, we saw how a lot of the functions we’ve been writing for different combinator libraries can be expressed in terms of a single interface, `Monad`. Monads provide a powerful interface, as evidenced by the fact that we can use `flatMap` to essentially write imperative programs in a purely functional way.

In this chapter, we’ll learn about a related abstraction, *applicative functors*, which are less powerful than monads, but more general (and hence more common). The process of arriving at applicative functors will also provide some insight into *how to discover such abstractions*, and we’ll use some of these ideas to uncover another useful abstraction, *traversable functors*. It may take some time for the full significance and usefulness of these abstractions to sink in, but you’ll see them popping up again and again in your daily work with FP if you pay attention.

12.1 Generalizing monads

By now we’ve seen various operations, like `sequence` and `traverse`, implemented many times for different monads, and in the last chapter we generalized the implementations to work for *any* monad `F`:

```
def sequence[A](lfa: List[F[A]]): F[List[A]]
  traverse(lfa)(fa => fa)

def traverse[A,B](as: List[A])(f: A => F[B]): F[List[B]]
  as.foldRight(unit(List[B]()))((a, mbs) => map2(f(a), mbs)(_ :: _))
```